

COUCHDB – APUNTE

Couch, al igual que Mongo, está orientada a documentos basados en JSON y REST. Una de las principales diferencias con Mongo son las queries ad-hoc. En su lugar, Couch tiene vistas indexadas producidas por operaciones mapreduce incrementales, que son la forma principal de descubrir documentos.

Está diseñada con la web en mente, por lo que su robustez es incomparable.

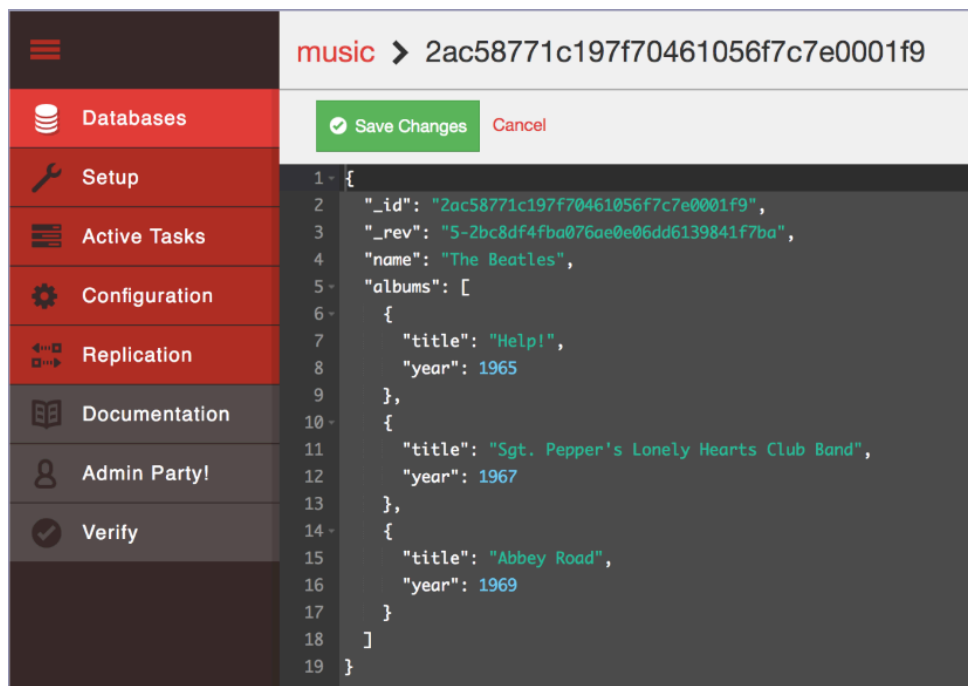
RELAXING ON THE COUCH

Apunta a trabajar bien y a soportar múltiples escenarios de despliegue (desde un teléfono Android hasta un datacenter entero).

La replicación puede ser en un sentido o bidireccional, también puede ser ad-hoc o continua, Couch ofrece mucha versatilidad.

DAY 1: CRUD, FAUXTON AND cURL Redux

Al estar basada en JSON's las modificaciones se pueden hacer de forma similar a Mongo. Se pueden realizar directamente desde la GUI Fauxton:



Como es esperable, el botón delete borra el documento de la base de datos.

PERFORMING RESTful CRUD OPERATIONS WITH cURL

La comunicación con Couch está basada en REST, lo que significa: comandos HTTP. Primero nos conectamos a la base de datos y verificamos que está todo ok:

```
$ export COUCH_ROOT_URL=http://localhost:5984
$ curl ${COUCH_ROOT_URL}
{
  "couchdb": "Welcome",
  "version": "2.0.0",
  "vendor": {
    "name": "The Apache Software Foundation"
  }
}
```

Para obtener información de la base de datos que acabamos de crear:

```
$ curl "${COUCH_ROOT_URL}/music/"
{
  "db_name": "music",
  "update_seq": "4-g1AA...aZxxw",
  "sizes": {
    "file": 24907,
    "external": 193,
    "active": 968
  },
  "purge_seq": 0,
  "other": {
    "data_size": 193
  },
  "doc_del_count": 0,
  "doc_count": 1,
  "disk_size": 24907,
  "disk_format_version": 6,
  "data_size": 968,
  "compact_running": false,
  "instance_start_time": "0"
}
```

READING A DOCUMENT WITH GET

Para recuperar un documento en específico, anexamos el `_id` al URL de la base de datos:

```
$ curl "${COUCH_ROOT_URL}/music/2ac58771c197f70461056f7c7e0001f9"
{
  "_id": "2ac58771c197f70461056f7c7e0001f9",
  "_rev": "8-elb7281f6adcd82910c6473be2d4e2ec",
  "name": "The Beatles",
  "albums": [
    {
      "title": "Help!",
      "year": 1965
    },
    {
      "title": "Sgt. Pepper's Lonely Hearts Club Band",
      "year": 1967
    },
    {
      "title": "Abbey Road",
      "year": 1969
    }
  ]
}
```

Usar peticiones `get` (es la elegida por defecto) es siempre seguro, ya que no se modifica la base de datos.

CREATING A DOCUMENT WITH POST

Para crear un documento usamos `POST`. Hay que especificar un encabezado de tipo de contenido con el valor `application/json`. El código `201` nos dice que la operación fue realizada con éxito.

```
$ curl -i -XPOST "${COUCH_ROOT_URL}/music/" \
-H "Content-Type: application/json" \
-d '{ "name": "Wings" }'
HTTP/1.1 201 Created
Cache-Control: must-revalidate
Content-Length: 95
Content-Type: application/json
Date: Sun, 30 Apr 2017 23:15:42 GMT
Location: http://localhost:5984/music/2ac58771c197f70461056f7c7e002eda
Server: CouchDB/2.0.0 (Erlang OTP/19)
X-Couch-Request-ID: 92885ae1d3
X-CouchDB-Body-Time: 0

{
  "ok": true,
  "id": "2ac58771c197f70461056f7c7e002eda",
  "rev": "1-2fe1dd1911153eb9df8460747dfe75a0"
}
```

UPDATING A DOCUMENT WITH PUT

PUT se utiliza para modificar un documento, o para crear uno con un `_id` específico:

```
$ curl -i -XPUT \
"${COUCH_ROOT_URL}/music/2ac58771c197f70461056f7c7e002eda" \
-H "Content-Type: application/json" \
-d '{
  "_id": "74c7a8d2a8548c8b97da748f43000f1b",
  "_rev": "1-2fe1dd1911153eb9df8460747dfe75a0",
  "name": "Wings",
  "albums": ["Wild Life", "Band on the Run", "London Town"]
}'
HTTP/1.1 201 Created
Cache-Control: must-revalidate
Content-Length: 95
Content-Type: application/json
Date: Sun, 30 Apr 2017 23:25:13 GMT
ETag: "2-17e4ce41cd33d6a38f04a8452d5a860b"
Location: http://localhost:5984/music/2ac58771c197f70461056f7c7e002eda
Server: CouchDB/2.0.0 (Erlang OTP/19)
X-Couch-Request-ID: 6c0bdfffa5
X-CouchDB-Body-Time: 0

{
  "ok": true,
  "id": "2ac58771c197f70461056f7c7e002eda",
  "rev": "2-17e4ce41cd33d6a38f04a8452d5a860b"
}
```

Tanto el campo `_id` como el `_rev` deben coincidir en el documento a modificar, de lo contrario la operación fallará.

REMOVING A DOCUMENT WITH DELETE

Para borrar un documento de la base de datos:

```
$ curl -i -XDELETE \
  "${COUCH_ROOT_URL}/music/2ac58771c197f70461056f7c7e002eda" \
  -H "If-Match: 2-17e4ce41cd33d6a38f04a8452d5a860b"
HTTP/1.1 200 OK
Cache-Control: must-revalidate
Content-Length: 95
Content-Type: application/json
Date: Sun, 30 Apr 2017 23:26:40 GMT
ETag: "3-42aafb7411c092614ce7c9f4ab79dc8b"
Server: CouchDB/2.0.0 (Erlang OTP/19)
X-Couch-Request-ID: c4dcb91db2
X-CouchDB-Body-Time: 0

{
  "ok": true,
  "id": "2ac58771c197f70461056f7c7e002eda",
  "rev": "3-42aafb7411c092614ce7c9f4ab79dc8b"
}
```

Vale aclarar que el documento no está “eliminado” del disco, en su lugar se adjuntó un nuevo documento vacío. Couch no modifica los documentos in place. Pero a fines prácticos, está eliminado.

DAY 2: CREATING AND QUERYING VIEWS

Una vista es una ventana a un documento en Couch

ACCESSING DOCUMENTS THROUGH VIEWS

Una vista consiste en funciones de mapeo y de reducción, usadas para generar una lista ordenada de pares clave-valor. La más utilizada es la vista llamada `_all_docs`. Contiene una entrada por cada documento en la base de datos:

```
$ curl "${COUCH_ROOT_URL}/music/_all_docs"
{
  "total_rows": 1,
  "offset": 0,
  "rows": [
    {
      "id": "2ac58771c197f70461056f7c7e0001f9",
      "key": "2ac58771c197f70461056f7c7e0001f9",
      "value": {
        "rev": "7-d37c47883f4d30913c6a38644410685d"
      }
    }
  ]
}
```

Por defecto, las vistas incluirán solo metadata para los documentos en el campo value en lugar de el contenido de cada documento. Para recuperar los campos de los documentos, adjuntamos el parámetro `include_docs = true` a la URL:

```
$ curl "${COUCH_ROOT_URL}/music/_all_docs?include_docs=true"
{
  "total_rows": 1,
  "offset": 0,
  "rows": [
    {
      "id": "2ac58771c197f70461056f7c7e0001f9",
      "key": "2ac58771c197f70461056f7c7e0001f9",
      "value": {
        "rev": "7-d37c47883f4d30913c6a38644410685d"
      },
      "doc": {
        "_id": "2ac58771c197f70461056f7c7e0001f9",
        "_rev": "7-d37c47883f4d30913c6a38644410685d",
        "name": "The Beatles",
        "albums": [
          {
            "title": "Help!",
            "year": 1965
          },
          {
            "title": "Sgt. Pepper's Lonely Hearts Club Band",
            "year": 1967
          },
          {
            "title": "Abbey Road",
            "year": 1969
          }
        ]
      }
    }
  ]
}
```

WRITING YOUR FIRST VIEW

Se hace principalmente mediante la UI Fauxton:

The screenshot shows the Fauxton web interface. On the left is a sidebar with navigation links: Databases, Setup, Active Tasks, Configuration, Replication, Documentation, Admin Party!, and Verify. The main area is titled 'music' and contains a list of database operations: All Documents, Run A Query with Mango, Permissions, Changes, and Design Documents. The 'Design Documents' option is selected, leading to the 'New View' dialog. This dialog has fields for 'Design Document' (set to 'New document'), 'Index name' (set to 'new-view'), and a 'Map function' (containing a JavaScript function that emits document IDs). There is also a 'Reduce (optional)' dropdown set to 'NONE'. At the bottom are two buttons: 'Create Document and Build Index' (green) and 'Cancel' (red).

Cambiando la función emit() podemos modificar la vista. Por ejemplo, si le pasamos los parámetros emit(null,doc) vamos a tener un output similar a este:

```
id "2ac58771c197f70461056f7c7e0001f9"

{
  "id": "2ac58771c197f70461056f7c7e0001f9",
  "key": null,
  "value": {
    "_id": "2ac58771c197f70461056f7c7e0001f9",
    "_rev": "8-e1b7281f6adcd82910c6473be2d4e2ec",
    "name": "The Beatles",
    "albums": [
      {
        "title": "Help!",
        "year": 1965
      },
      {
        "title": "Sgt. Pepper's Lonely Hearts Club Band",
        "year": 1967
      },
      {
        "title": "Abbey Road",
        "year": 1969
      }
    ]
  },
  "_id": "2ac58771c197f70461056f7c7e0001f9"
}
```

SAVING A VIEW AS A DESIGN DOCUMENT

En producción las vistas deben ser guardadas en documentos de diseño. Son documentos reales en la base de datos.

Sus ID's empiezan con `_design/` y contienen una o más `views`. Se debe agrupar vistas en base a lo que realizan relativo a nuestra información.

FINDING ARTISTS BY NAME

Usando la vista `_all_docs` podemos acceder a los documentos por su ID, pero ¿Y si queremos acceder por el nombre de los artistas?

En Fauxton, en la página para crear una nueva vista, colocamos la siguiente función de mapeo:

```
couchdb/artistsByNameMapper.js
function(doc) {
  if ('name' in doc) {
    emit(doc.name, doc._id);
  }
}
```

Esta función revisa si el documento actual tiene un campo "name" y, si lo tiene, emite el nombre y el id del documento.

FINDING ALBUMS BY NAME

Nuevamente, en la página para crear una nueva lista en Fauxton:

```
couchdb/albumsByNameMapper.js
function(doc) {
  if ('name' in doc && 'albums' in doc) {
    doc.albums.forEach(function(album){
      var
        key = album.title || album.name,
        value = { by: doc.name, album: album };
      emit(key, value);
    });
  }
}
```


Esta función devuelve:

Key	Value
"Abbey Road"	{by: "The Beatles", album: {title: "Abbey Road", year: 1969}}
"Help!"	{by: "The Beatles", album: {title: "Help!", year: 1965}}
"Sgt. Pepper's Lonely Hearts Club Band"	{by: "The Beatles", album: {title: "Sgt. Pepper's Lonely Hearts Club Band", year: 1967}}

QUERYING OUR CUSTOM ARTIST AND ALBUM VIEWS

De vuelta a la línea de comandos para realizar queries con el comando curl. vamos a comenzar con la vista de artistas por nombre:

```
$ curl "${COUCH_ROOT_URL}/music/_design/artists/_view/by_name"
{
  "total_rows": 1,
  "offset": 0,
  "rows": [
    {
      "id": "2ac58771c197f70461056f7c7e0001f9",
      "key": "The Beatles",
      "value": "2ac58771c197f70461056f7c7e0001f9"
    }
  ]
}
```

Ahora intentemos buscar álbumes por nombre:

```

$ curl "${COUCH_ROOT_URL}/music/_design/albums/_view/t
{
  "total_rows": 3,
  "offset": 0,
  "rows": [
    {
      "id": "2ac58771c197f70461056f7c7e0001f9",
      "key": "Abbey Road",
      "value": {
        "by": "The Beatles",
        "album": {
          "title": "Abbey Road",
          "year": 1969
        }
      }
    },
    {
      "id": "2ac58771c197f70461056f7c7e0001f9",
      "key": "Help!",
      "value": {
        "by": "The Beatles",
        "album": {
          "title": "Help!",
          "year": 1965
        }
      }
    },
    {
      "id": "2ac58771c197f70461056f7c7e0001f9",
      "key": "Sgt. Pepper's Lonely Hearts Club Band",
      "value": {
        "by": "The Beatles",
        "album": {
          "title": "Sgt. Pepper's Lonely Hearts Club Band",
          "year": 1967
        }
      }
    }
  ]
}

```

Couch se va a encargar de que las llaves se muestren en orden alfanumérico. Ese es el tipo de indexado que Couch ofrece.

Cuando se especifica una llave, solo las filas con esa llave exacta serán retornadas:

```
$ curl '${COUCH_ROOT_URL}/music/_design/albums/_view/by_name?key="Help!"'
{
  "total_rows": 3,
  "offset": 1,
  "rows": [
    {
      "id": "2ac58771c197f70461056f7c7e0001f9",
      "key": "Help!",
      "value": {
        "by": "The Beatles",
        "album": {
          "title": "Help!",
          "year": 1965
        }
      }
    }
  ]
}
```

El número de indentaciones nos sugieren que tan “profundo” llegó la query en documentos anidados.

IMPORTING DATA INTO COUCHDB USING RUBY

Primero descargamos la versión comprimida de GZIP desde Jamendo:

```
$ curl -O https://imgjam.com/data/dbdump_artistalbumtrack.xml.gz
```

Para parsear el archivo XML usaremos la gem libxml-ruby y la gem couchrest:

```
$ gem install libxml-ruby couchrest
```

Vamos a usar un parseador para procesar documentos de forma secuencial:

```
couchdb/import_from_jamendo.rb
```

```
require 'libxml'
```

```
require 'couchrest'
```

```
include LibXML
```

```
class JamendoCallbacks
```

```
  include XML::SaxParser::Callbacks
```

```
  def initialize
```

```
    @db = CouchRest.database!("http://localhost:5984/music")
```

```
    @count = 0
```

```
    @max = 10000 # maximum number to insert
```

```
    @stack = []
```

```
    @artist = nil
```

```
    @album = nil
```

```
    @track = nil
```

```
    @tag = nil
```

```
    @buffer = nil
```

```
  end
```

```

def on_start_element(element, attributes)
  case element
  when 'artist'
    @artist = { :albums => [] }
    @stack.push @artist
  when 'album'
    @album = { :tracks => [] }
    @artist[:albums].push @album
    @stack.push @album
  when 'track'
    @track = { :tags => [] }
    @album[:tracks].push @track
    @stack.push @track
  when 'tag'
    @tag = {}
    @track[:tags].push @tag
    @stack.push @tag
  when 'Artists', 'Albums', 'Tracks', 'Tags'
    # ignore
  else
    @buffer = []
  end
end

def on_characters(chars)
  @buffer << chars unless @buffer.nil?
end

def on_end_element(element)
  case element
  when 'artist'
    @stack.pop
    @artist['_id'] = @artist['id'] # reuse Jamendo's artist id for doc_id
    @artist[:random] = rand
    @db.save_doc(@artist, false, true)
    @count += 1
    if !@max.nil? && @count >= @max
      on_end_document
    end
    if @count % 500 == 0
      puts " #{@count} records inserted"
    end
  when 'album', 'track', 'tag'
    top = @stack.pop
    top[:random] = rand
  when 'Artists', 'Albums', 'Tracks', 'Tags'
    # ignore
  else
    if @stack[-1] && @buffer
      @stack[-1][element] = @buffer.join.force_encoding('utf-8')
      @buffer = nil
    end
  end
end

```

```

        end
    end
end

def on_end_document
    puts "TOTAL: #{@count} records inserted"
    exit(0)
end
end

```

Para correr el script:

```

$ zcat < dbdump_artistaalbumtrack.xml.gz | ruby import_from_jamendo.rb
TOTAL: 10000 records inserted

```

Ahora, haremos consultas. Nos traeremos algunos artistas (limitados por el parámetro limit en el URL):

```

$ curl "${COUCH_ROOT_URL}/music/_design/artists/_view/by_name?limit=5"
{"total_rows":10001,"offset":0,"rows":[
{"id":"5385","key":" A.n.K.h // ","value":"5385"},
{"id":"354581","key":" E2U","value":"354581"},
{"id":"457184","key":" EL VECINO","value":"457184"},
{"id":"338059","key":" ENIGMA63","value":"338059"},
{"id":"378976","key":" John Ov3rblast","value":"378976"}
]}

```

Para pasar al medio de la lista usamos el parámetro startkey en el URL:

```

$ curl "${COUCH_ROOT_URL}/music/_design/artists/_view/by_name?\
limit=5&startkey=%22C%22"
{"total_rows":10001,"offset":1320,"rows":[
{"id":"363267","key":"C-74","value":"363267"},
{"id":"357962","key":"c-dio","value":"357962"},
{"id":"350911","key":"C-Jay L'infidel J.B","value":"350911"},
{"id":"1188","key":"c-nergy","value":"1188"},
{"id":"832","key":"C. Glen Williams","value":"832"}
]}

```

También podemos especificar que nos traiga los artistas entre la C y la D:

```
$ curl "${COUCH_ROOT_URL}/music/_design/artists/_view/by_name?\\
startkey=%22C%22&endkey=%22D%22&limit=5"
{"total_rows":10001,"offset":1320,"rows":[
{"id":"363267","key":"C-74","value":"363267"},
{"id":"357962","key":"c-dio","value":"357962"},
{"id":"350911","key":"C-Jay L'infidel J.B","value":"350911"},
{"id":"1188","key":"c-nergy","value":"1188"},
{"id":"832","key":"C. Glen Williams","value":"832"}
]}
```

Para hacerlo en orden inverso:

```
$ curl "${COUCH_ROOT_URL}/music/_design/artists/_view/by_name?\\
startkey=%22D%22&endkey=%22C%22&limit=5&descending=true"
{"total_rows":10001,"offset":8174,"rows":[
{"id":"1689","key":"czskamaarù","value":"1689"},
{"id":"341426","key":"CZAQU","value":"341426"},
{"id":"360640","key":"Cystoflo","value":"360640"},
{"id":"355941","key":"CYRUS DA VIRUS","value":"355941"},
{"id":"427004","key":"Cyrrix Project","value":"427004"}
]}
```

DAY 3: ADVANCED VIEWS, CHANGES API AND REPLICATING DATA

CREATING ADVANCED VIEWS WITH REDUCERS

Las Advanced Views amplían las capacidades de las vistas básicas de CouchDB, permitiendo realizar consultas más sofisticadas sobre los documentos almacenados. Mediante funciones map es posible generar índices personalizados, filtrar información y organizar los resultados según distintos criterios. Estas vistas se almacenan y actualizan de manera incremental, lo que permite realizar consultas eficientes sin tener que recorrer toda la base de datos cada vez.

Los Reducers complementan a las funciones map permitiendo resumir y agregar la información generada por las vistas. A partir de los pares clave-valor producidos por la fase map, las funciones reduce pueden realizar operaciones como conteos, sumas o agrupaciones, obteniendo estadísticas y resúmenes de grandes conjuntos de documentos. De esta forma, CouchDB implementa un modelo de consultas analíticas basado en MapReduce en lugar de utilizar agregaciones SQL tradicionales.

WATCHING COUCHDB FOR CHANGES

La Changes API provee mecanismos para vigilar a la base de datos y aplicar actualizaciones de forma inmediata ante cambios

Utilizaremos [Node.js](#) porque, al estar basado en eventos, nos permitirá ejemplificar de forma efectiva la API de cambios.

cURLing FOR CHANGES

La primera forma para acceder a la API de cambios es mediante la polling interface:

```
$ curl "${COUCH_ROOT_URL}/music/_changes"
{
  "results":[{
    "seq":"10057-g1....FqzAI2DMmw",
    "id":"370255",
    "changes":[{"rev":"1-a7b7cc38d4130f0a5f3eae5d2c963d85"}]
  },{
    "seq":"10057-g1....A0Y9NEs7RUB",
    "id":"370254",
    "changes":[{"rev":"1-2c7e0deec3ffca959ba0169b0e8bfcef"}]
  },{
    ... many more records ...
  },{
    "seq":"10057-g1....U90zMnILy7J",
    "id":"357995",
    "changes":[{"rev":"1-aa649aa53f2858cb609684320c235aee"}]
  }],
  "last_seq":100
}
```

Si no se especifica ningún parámetro, Couch devuelve todos los cambios que se han realizado en la base de datos. Podemos filtrar estos datos con parámetros en la URL (en este ejemplo usamos since):

```
$ curl "${COUCH_ROOT_URL}/music/_changes?since=10057-g1....FqzAI2DMmw"
{
  "results":[{
    "seq":"10057-g1....U90zMnILy7J",
    "id":"357995",
    "changes":[{"rev":"1-aa649aa53f2858cb609684320c235aee"}]
  }],
  "last_seq":100
}
```


Usando este método, la aplicación cliente estará chequeando periódicamente por cambios, tomando acciones específicas de la aplicación en cuestión.

Polling puede ser una solución si no se producen cambios muy a menudo en la base de datos. Si se quiere actualizaciones más rápidas sin tener overhead o reabrir conexiones, long polling es una mejor opción. Se especifica como un parámetro en la URL:

```
$ curl "${COUCH_ROOT_URL}/music/_changes?feed=longpoll&\
since=10057-g1....FqzAI2DMmw"
{"results":[
```

La principal diferencia está en el tiempo que Couch está dispuesto a mantener la conexión abierta.

POLLING FOR CHANGES WITH [Node.js](#)

Primeramente haremos una mirada preliminar de nuestro programa “observador”:

couchdb/watchChangesSkeleton.js

```
var http = require('http'),
    events = require('events');

/**
 * create a CouchDB watcher based on connection criteria;
 * follows the Node.js EventEmitter pattern, emits 'change' events.
 */
exports.createWatcher = function(options) {
  var watcher = new events.EventEmitter();

  watcher.host = options.host || 'localhost';
  watcher.port = options.port || 5984;
  watcher.last_seq = options.last_seq || 0;
  watcher.db = options.db || '_users';

  watcher.start = function() {
    // ... feed-specific implementation ...
  };

  return watcher;
};

// start watching CouchDB for changes if running as main script
if (!module.parent) {
  exports.createWatcher({
    db: process.argv[2],
    last_seq: process.argv[3]
  })
  .on('change', console.log)
  .on('error', console.error)
  .start();
}
```

Aún no colocamos nada de código perteneciente a Couch.
Ahora vamos a adjuntar el código para conectar con Couch
via long polling y emitir eventos:

couchdb/watchChangesLongpollingImpl.js

```
var httpOptions = {
  host: watcher.host,
  port: watcher.port,
  path: '/' +
    watcher.db +
    '/_changes' +
    '?feed=longpoll&include_docs=true&since=' +
    watcher.last_seq
};

http.get(httpOptions, function(res) {
  var buffer = '';

  res.on('data', function (chunk) {
    buffer += chunk;
  });
  res.on('end', function() {
    var output = JSON.parse(buffer);
    if (output.results) {
      watcher.last_seq = output.last_seq;
      output.results.forEach(function(change){
        watcher.emit('change', change);
      });
      watcher.start();
    } else {
      watcher.emit('error', output);
    }
  })
})
.on('error', function(err) {
  watcher.emit('error', err);
});
```

Para correr este script en la línea de comandos:

```
$ node watchChangesLongpolling.js music
{ seq: '...',
  id: '370255',
  changes: [ { rev: '1-a7b7cc38d4130f0a5f3eae5d2c963d85' } ],
  doc:
    { _id: '370255',
      _rev: '1-a7b7cc38d4130f0a5f3eae5d2c963d85',
      albums: [ [Object] ],
      id: '370255',
      name: '"ATTIC"',
      url: 'http://www.jamendo.com/artist/ATTIC_(3)',
      mbgid: '',
      random: 0.4121620435325435 } }
{ seq: '...',
  id: '370254',
  changes: [ { rev: '1-2c7e0deec3ffca959ba0169b0e8bfcef' } ],
  doc:
    { _id: '370254',
      _rev: '1-2c7e0deec3ffca959ba0169b0e8bfcef',
      ... many more entries ...
```

WATCHING FOR CHANGES CONTINUOUSLY

El feed continuos, en lugar de combinar todos los cambios disponibles en un arreglo de resultados, manda cada cambio de forma separada y mantiene la conexión abierta:

```
$ curl "${COUCH_ROOT_URL}/music/_changes?since=...feed=continuous"
{"seq":"...", "id":"357999", "changes":[{"rev":"1-0329f5c885...87b39beab0"}]}
{"seq":"...", "id":"357998", "changes":[{"rev":"1-79c3fd2fe6...1e45e4e35f"}]}
{"seq":"...", "id":"357995", "changes":[{"rev":"1-aa649aa53f...320c235aee"}]}
```

Eventualmente, si no se detectan cambios en un período de tiempo determinado, se cerrará la conexión.

No es una buena opción si tu cliente es un navegador web. Un navegador descargando la información de manera asíncrona puede no recibir nada de información hasta que toda la conexión finalice.

FILTERING CHANGES

Se aplican filter functions para filtrar solo los cambios requeridos. Una función de filtro puede verse de la siguiente manera:

```
function(doc) {
  return doc.country === "RUS";
}
```

También se pueden parametrizar para lograr mayor generalidad:

```
function(doc, req) {
  return doc.country === req.query.country;
}
```

Para ver esto en acción creamos el filtro:

```
$ curl -XPUT "${COUCH_ROOT_URL}/music/_design/wherabouts" \
-H "Content-Type: application/json" \
-d '{"language":"javascript","filters":{"by_country":
  "function(doc,req){return doc.country === req.query.country;}"
  }'}
{
  "ok":true,
  "id":"_design/wherabouts",
  "rev":"1-c08b557d676ab861957eae85b628d74"
}
```

Finalmente, podemos hacer una petición con los cambios filtrados:

```
$ curl "${COUCH_ROOT_URL}/music/_changes?\
filter=wherabouts/by_country&\
country=RUS"
{"results":[
{"seq":10,"id":"5987","changes":[{"rev":"1-2221be...a3b254"}]},
{"seq":57,"id":"349359","changes":[{"rev":"1-548bde...888a83"}]},
{"seq":73,"id":"364718","changes":[{"rev":"1-158d2e...5a7219"}]},
...
}
```

Son versátiles debido a que las funciones contienen código en JavaScript, por lo que es altamente modificable.

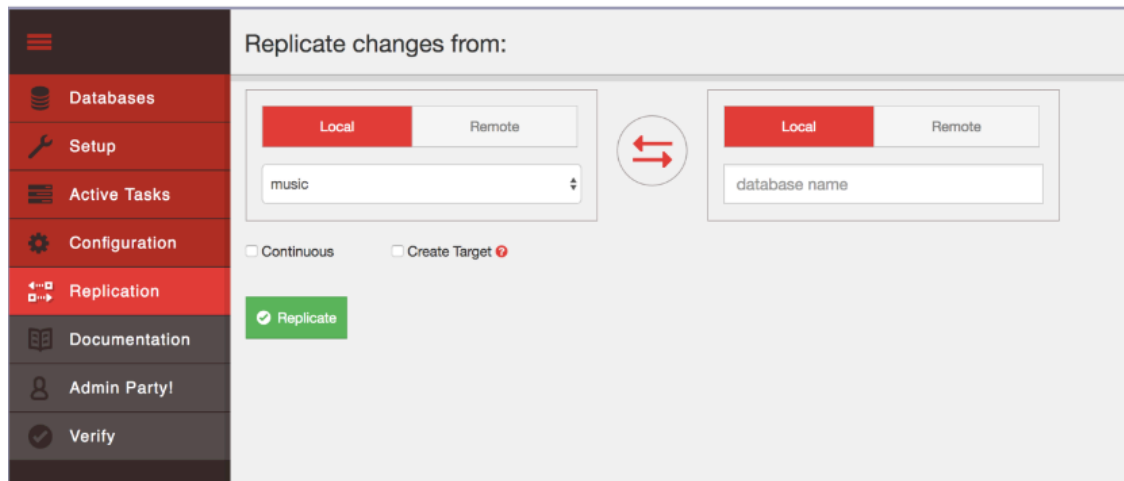
REPLICATING DATA IN COUCHDB

Según Couch, la forma más segura de guardar información es en varios nodos en tu cluster, y Couch provee las herramientas para hacerlo. Couch tiene replicación master-master. Cada servidor puede recibir

actualizaciones, responder a requests y eliminar data sin importar si se puede conectar o no a otro servidor.

En este modelo, los cambios son selectivamente replicados en una dirección y toda la información está sujeta a replicación en el mismo sentido.

Usaremos nuevamente nuestra GUI:



Veremos un gran cartel verde confirmando que la solicitud de replicación ha comenzado.

Ahora deberíamos tener 2 bases de datos. Una “music” y otra “musci-repl”.

CREATING CONFLICTS

Crearemos conflictos para ver cómo lidiar con ellos:

```
$ curl -XPUT "${COUCH_ROOT_URL}/music/theconflicts" \
-H "Content-Type: application/json" \
-d '{ "name": "The Conflicts" }'
{
  "ok":true,
  "id":"theconflicts",
  "rev":"1-e007498c59e95d23912be35545049174"
}
```

Volvemos a accionar otra sincronización y chequeamos en music-repl:

```
$ curl -XPUT "${COUCH_ROOT_URL}/music-repl/theconflicts" \
-H "Content-Type: application/json" \
-d '{
  "_id": "theconflicts",
  "_rev": "1-e007498c59e95d23912be35545049174",
  "name": "The Conflicts",
  "albums": ["Conflicts of Interest"]
}'

{
  "ok":true,
  "id":"theconflicts",
  "rev":"2-0c969fbfa76eb7fcdf6412ef219fcac5"
}
```

Y ahora inducimos un conflicto añadiendo el álbum “Conflicting Opinions”:

```
$ curl -XPUT "${COUCH_ROOT_URL}/music/theconflicts" \
-H "Content-Type: application/json" \
-d '{
  "_id": "theconflicts",
  "_rev": "1-e007498c59e95d23912be35545049174",
  "name": "The Conflicts",
  "albums": ["Conflicting Opinions"]
}'

{
  "ok":true,
  "id":"theconflicts",
  "rev":"2-cab47bf4444a20d6a2d2204330fdce2a"
}
```

En este punto, ambos documentos están en la versión 2 y derivan de la misma revisión base -> conflicto ?.

RESOLVING CONFLICTS

No, la operación fue realizada con éxito. Couch elige una de las bases de datos usando un algoritmo determinista. Couch guarda la opción “perdedora” por las dudas de que se necesite intervención manual luego.

Para ver qué versión “ganó” usamos el parámetro conflicts=true en el URL en una petición GET normal:

```
$ curl "${COUCH_ROOT_URL}/music-repl/theconflicts?conflicts=true"
{
  "_id": "theconflicts",
  "_rev": "2-cab47bf4444a20d6a2d2204330fdce2a",
  "name": "The Conflicts",
  "albums": ["Conflicting Opinions"],
  "_conflicts": [
    "2-0c969fbfa76eb7fcdf6412ef219fcac5"
  ]
}
```

Aquí vemos que la segunda actualización ganó.

Añadiendo un parámetro rev a la petición GET podemos traernos estas revisiones conflictivas y decidir que hacer con ellas:

```
$ curl "${COUCH_ROOT_URL}/music-repl/theconflicts?rev=2-0c969f..."
{
  "_id": "theconflicts",
  "_rev": "2-0c969fbfa76eb7fcdf6412ef219fcac5",
  "name": "The Conflicts",
  "albums": ["Conflicts of Interest"]
}
```